California State University, Channel Islands
Department of Computer Science
COMP 569 – Project 1 Technical Guide

This document provides a comprehensive technical and conceptual explanation of Project 1. It is intended to support students in understanding the mathematical foundations, algorithmic design decisions, and real-world relevance of informed search and constraint satisfaction. The content in this guide is explanatory in nature and is not meant to prescribe a single implementation. Students are encouraged to explore alternative designs as long as the core project requirements are met.

Part A: Informed Search for Autonomous Navigation

Autonomous navigation is one of the most common applications of artificial intelligence. In this project, the navigation problem is modeled as a state-space search problem where an agent must move through an environment while minimizing cost and avoiding obstacles.

A.1 State-Space Representation

The navigation problem is modeled using a **state-space representation**, which provides a mathematical abstraction of the environment and the agent's possible actions.

The search problem is defined as the tuple:

$$\langle S, A, T, C, G \rangle$$

where:

- **S** — the set of all possible states
- **A** — the set of available actions
- **T(s, a) → s′** — the transition function that maps a state and action to a new state
- **C(s, a, s′)** — the cost associated with taking an action
- **G** — the set of goal states

State Representation

In this project, each state is defined as:

$$s = (x, y, z)$$

where:

- **x, y** represent the drone's horizontal position
- **z** represents the drone's altitude

This three-dimensional state representation captures real-world navigation constraints such as elevation changes, obstacle avoidance, and energy consumption, making it well suited for modeling autonomous drone movement in an urban environment.

A.2 Transition and Cost Model

Each action transitions the drone to a neighboring state. The cost function is designed to model energy consumption, which is higher when the drone gains altitude.

$C(s, a, s') =$

- 1 for horizontal movement
- 2 for upward movement
- 1 for downward movement

This asymmetric cost structure introduces realism and forces the search algorithm to balance distance minimization with energy efficiency.

A.3 Uniform Cost Search

Uniform Cost Search (UCS) expands nodes in order of increasing path cost g(n). It guarantees an optimal solution when all costs are non-negative.

$$g(n) = \Sigma\, C(s\_i, a\_i, s\_\{i+1\})$$

While optimal, UCS can be computationally expensive in large state spaces, making it a useful baseline for comparison rather than a practical solution.

A.4 A* Search

A* improves upon UCS by incorporating a heuristic estimate of remaining cost. The evaluation function is defined as:

$$f(n) = g(n) + h(n)$$

where g(n) is the known path cost and h(n) is the heuristic estimate.

A* is optimal if the heuristic is admissible and consistent, making it one of the most widely used algorithms in robotics and navigation.

A.5 Heuristic Design

Heuristics play a critical role in determining the efficiency of A* search.

Manhattan Distance:

$$h(n) = |x - x\_g| + |y - y\_g| + |z - z\_g|$$

Euclidean Distance:

$$h(n) = \sqrt{((x - x\_g)^2 + (y - y\_g)^2 + (z - z\_g)^2)}$$

Building-Aware Heuristic:
$h(n) = d_{xy} + \max(0, h_{obs} - z)$

This heuristic accounts for obstacle height and encourages energy-efficient routing.

A.6 Admissibility and Consistency
A heuristic h(n) is admissible if it never overestimates the true cost to the goal:

$h(n) \leq h^*(n)$
Consistency (or monotonicity) requires:
$h(n) \leq c(n, n') + h(n')$

These properties ensure that A* remains both optimal and efficient.

A.7 Practical Considerations
In practice, search performance depends on:

- Grid resolution
- Obstacle density
- Heuristic quality
- Branching factor

Students are encouraged to experiment with different heuristics and compare runtime, memory usage, and solution quality.

Part B: Constraint Satisfaction Problems

Constraint Satisfaction Problems (CSPs) represent a different class of AI problems where the objective is to find a valid assignment rather than an optimal path.

B.1 CSP Formulation
A CSP is formally defined as: $\langle X, D, C \rangle$

where:
• $X = \{X_1, X_2, \ldots, X_n\}$ are variables
• $D = \{D_1, D_2, \ldots, D_n\}$ are domains
• C is a set of constraints

A solution assigns a value to every variable such that all constraints are satisfied.

B.2 Workforce Scheduling as a CSP
In this project, variables represent tasks, and domains represent available technicians. Constraints ensure that technicians have the required skills, are available, and are not assigned to overlapping tasks.

B.3 Backtracking Search
Backtracking systematically explores assignments and backtracks when a constraint violation is detected. The algorithm follows a depth-first strategy:

1. Select an unassigned variable
2. Assign a value from its domain
3. Check constraints
4. Recurse or backtrack

This process continues until a complete assignment is found or all possibilities are exhausted.

B.4 Constraint Propagation
Constraint propagation improves efficiency by removing inconsistent values early. Forward checking removes values that violate constraints immediately after assignment, while arc consistency ensures all variable pairs remain compatible.

B.5 Computational Complexity
CSPs are NP-complete in general. Practical performance depends on:

- Domain size
- Constraint tightness
- Variable ordering
- Use of heuristics such as MRV and degree heuristics

These strategies significantly reduce search complexity in practice.

Conclusion

This project integrates two foundational AI paradigms: heuristic search and constraint reasoning. Together, they illustrate how intelligent systems make decisions, optimize outcomes, and operate under real-world constraints. The concepts explored in this project form the foundation for more advanced AI topics such as planning, robotics, and reinforcement learning.